Proceedings of the Sixth International Conference on Computing Methodologies and Communication (ICCMC 2022)
IEEE Xplore Part Number: CFP22K25-ART; ISBN: 978-1-6654-1028-1

2022 6th International Conference on Computing Methodologies and Communication (ICCMC) | 978-1-6654-1028-1/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICCMC53470.2022.9753800

A Search-Based Approach on Metaheuristic Algorithm for Software Modularization to optimize Software Modularity

Divya Sharma[1], Shikha Lohchab[2]
[1]Research Scholar, GD Goenka University, M.Tech (Computer Science), Amity University
Email Id: divya.07sharma@gmail.com

[2]Ph.D., Assistant Professor, GD Goenka University

Abstract— Software refactoring is a process to maintain software quality that will further improvise the software's internal structure without amending its external behaviour. The software refactoring is implemented in the software maintenance phase by modularizing the source code of the original Software clustering is a modularization approach that is used to modularize source code objects with the purpose of boosting the code's reusability and readability. Owing to the clustering issue being NP-hard, evolutionary methods like the genetic algorithm were utilized to address it. There is no search-based technique for modularization that uses a hierarchical method in the structural refactoring literature. It is an unsubstantiated learning technique utilized to cluster software objects (for example files, classes, or modules) with similar structures. The attained clusters can be utilized for a research study, analysis, and comprehending the software objects' structure and behaviour. While one observation is that executing software module clustering with optimum consequences is thought-provoking. In the research paper, we have utilized the local and global methods wherein a metaheuristic hierarchal search-based clustering approach has been introduced that can help in modularizing the software system. In the algorithm, the outcome is a tree that has nodes including artifacts wherein sub trees collate these artifacts and it is a cluster that is a candidate solution. This algorithm will help in extracting a newer model of the source code which will further help in deciding the correct artifacts which will redirect to obtain documents, packages, and components. The source code can be fetched from GitHub.

Keywords—Software module clustering, clustering algorithms, clustering estimation, refactoring, optimization, modularization, genetic algorithm, restructuring.

I. INTRODUCTION

The procedure of changing a software process after it was released to decrease the defects, expand the software performance, or improvise the software strategy is known as software maintenance. Its modules are significant for upcoming software growth and it around in takes 80% of the overall cost [1].

The software systems are planned and designed with disintegrating their complete assembly into smaller autonomous components or modules [35]. Such breakdown aids in falling the arrangement density and consequently advances the software quality of the product. Classes predict the part of modules in an "Object-Oriented" (OO) software process that summaries the approaches as well as variables. For complicated and large object-oriented processes, it was stated that a module may indicate the character of a component that clusters a group of classes collected to deliver well-defined facilities to the remaining of the classification [40]. It was analysed that a software program containing modules that display high cohesion and low coupling is simpler to comprehend and manage [21].

Some variations in software maintenance such as addition, deletion, or amendment code directly to the code chunks affect the development and future effort in code comprehensibility. Code smells are included in the source code which may hamper the process of software development [4].

Software refactoring is the process of adapting code for the source to correct the code smells deprived of any alteration within the process's exterior behavior. It improvises the source code software quality by dropping the possible number

of defects and this will further make the code simpler to sustain or encompass in the future. In their book [3], fowler et al. stated certain potential code smells for "Object-Oriented Programming" (OOP) based platforms and suggested probable refactoring situations. Since then, a lot of research was executed to offer new refactoring states or collateral consequences of using different situations in the source code to improve quality. There are two types of refactoring methods: structural and conceptual. For instance, retitle technique refactoring is a theoretical situation that deviates the method for a newer description of its task. Few of the structural refactoring methods are into approaches or functions composition.

Another structural refactoring methods are to improvise the issues in the code blocks. For instance, move technique factoring is a process that is well known as the piece of affecting a technique from one class to other that is co-related with that method. +e correspondence amongst methods may be structural relationships such as semantic or call relations. +ere are also a few of the compound refactoring approaches that are in relation to the set of basic methods that imitate complicated changes. To exemplify a process of structural refactoring, Fig 1 portrays an instance of re-modularization for a software product.

A node represents a class, whereas edges characterize a relation between classes in this figure. +ese classes are detached into 2 modules rendering to its associations. Fig 2 demonstrates various amendments to the software following some of the software maintenance phases. As revealed, the correspondence between the nodes is amended, and a novel class "I" is introduced to the scheme. In fig 2, the "G" node has a stronger correlation with the nodes in the left module as compared to the nodes in the right module. As a result, remodularization is required to change this node's location (and node "I"). +e remodularization result is revealed in Fig 3.

Present clustering approaches for finding appropriate modularization are grounded on 2basic hierarchical or non-hierarchical approaches. In A tree of links is created as a consequence of hierarchical techniques, beginning with the artefact at the leaf node and terminating at the root. +ese methods provide programmers with a hierarchical interpretation of the number and suitable cut point in a tree to concept modules.

The major disadvantages of +e in hierarchical algorithms are mentioned below [18]:

(1) Owing to the occurrence of "zigzag", it is required to create the entire tree to the conclusion in order to classify modules.

(2) +ere occurs no well-known opinion to choose in which the clustering process must end.

(3) Hierarchical clustering approaches are limited by arbitrary choices. +ese choices get a superb influence on the results of clustering. When confronted with arbitrary conclusions and an incorrect decision, there is no way to undo or modify the wrong decisions.

(4) +ese techniques are greedy and henceforth could not investigate the issue space well. Numerous previous research [10–11] have exposed these systems do not accomplish well in software clustering.

Even after a substantial development held in remodularization, the majority of research studies concentrate on improvising the quality of modularity (for example cohesion and coupling) of the software product to the greatest extent, without seeing the difference between present and formed modular structure. These methods might be valuable when the quality of the software has declined to the point where additional working with the classification is not feasible and the process desires through renovating.

Software remodularization with the least probable agitation may also be attained by using the restraints on the program of classes amongst the packages, but this method could result in the direction of a suboptimal explanation. The key benefit of using the fitness function of "PMQ" is that it assists in discovering each viable solution inside the search space. Furthermore, "PMQ" aids in directing the "search-based meta-heuristic" methods to a high-quality result by reducing amendments in the initial modular construction. To validate this supposition, "PMQ" is assessed with the "Simple Genetic Algorithm"(SGA)and "Multi-Objective Genetic Algorithm" (MGA such as NSGA-II: "Non-Dominated Sorting Genetic Algorithm") presented in Deb et a1.[19]. We select such meta heuristic methods specifically since they were utilized to resolve comparable software remodularization glitches in the literature [1, 16].

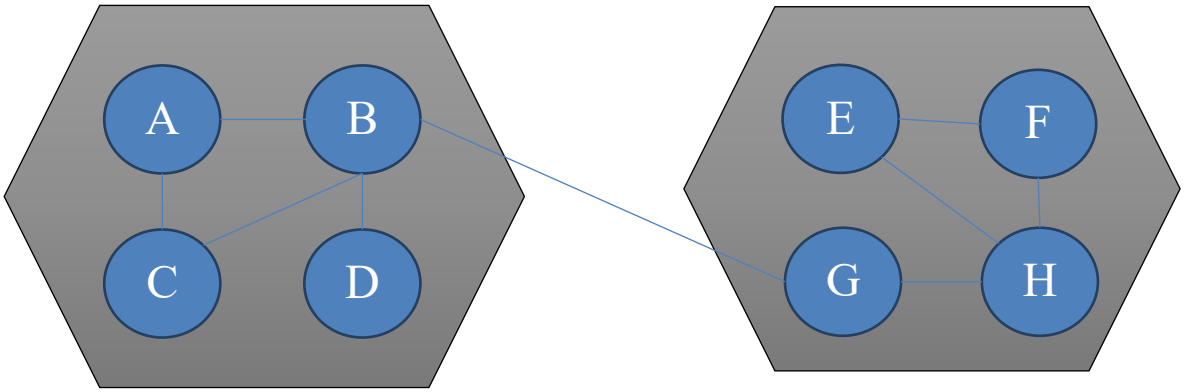

Fig. 1: Modularization for a Software Process

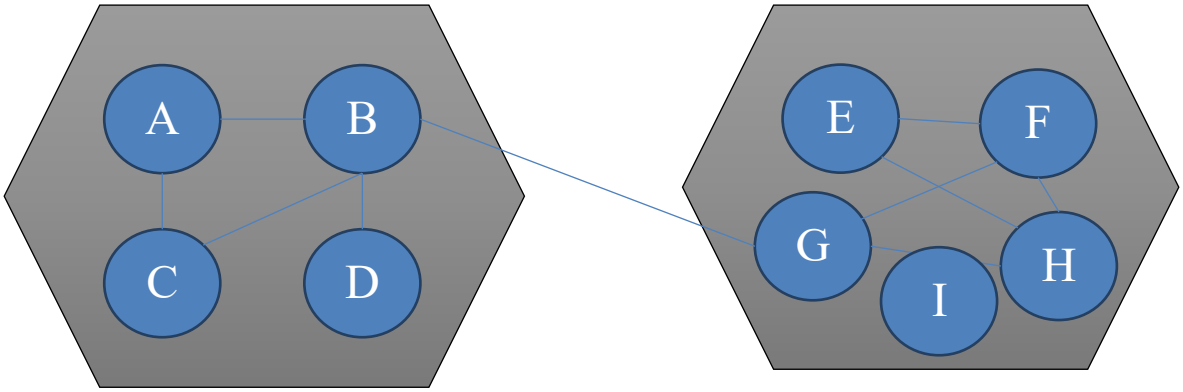

Fig.2: Few maintenance tasks for Figure 1

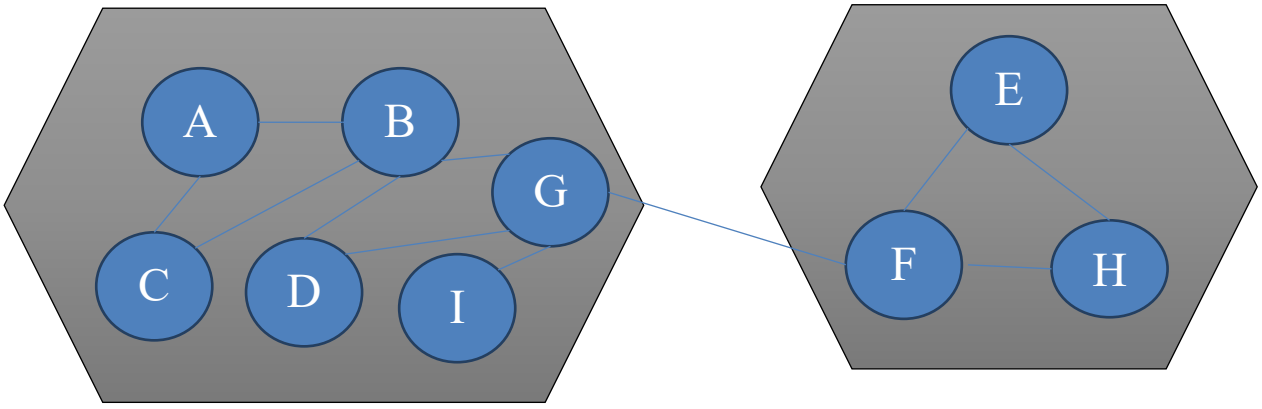

Fig.3:Remodularization executed in Figure 2

Other meta heuristics may be employed to assess the PMQ in addition to the genetic algorithm. Nevertheless, they would require a significant quantity of time for constraint tuning, and if the variables are not adjusted correctly, the produced outcome might be a poor resolution. The benefit of employing the previously described Genetic Algorithm (GA)-based optimization system is that its variable standards were tuned via prior studies.

The primary aids of this article are briefly described as follows:

(1) The research article introduces a search-based optimization method to the software issue for improvising the modular assembly of a current object-oriented software package group with the least possible agitation.

(2) To direct the search techniques near a solution with better quality with the least potential alteration, a new fitness function, called PMQ, was introduced.

(3) To authorize the PMQ fitness function's supremacy, it was presented and enhanced with the MGA and SGA to speech the remodularization issue.

(4) An empirical analysis is led to assess the efficiency of the suggested technique on 5 real-world as well as 3 random systems.

(5) The following are the research study's main findings:

(a) The suggested technique with disciplined optimization is able to attain the parallel MQ level with the least disturbance with worldwide optimization in single as well as multi-objective genetic algorithms.

(b)The MGA achieves improvement for global and penalized optimization than SGA.

Related Work

Several researchers have started to improve the perceptions of orientation, and an automated answer for sleuthing and revamping code smells like [17–20] by Following the publication of Fowler's book [3] A method for structural refactoring is remodularization of source code objects. Several search-based research was conducted due to the huge resolution space for modularization.

A GA, especially Bunch-GA, as well as 2 hill-climbing methods i.e., Bunch-SAHC and NAHC, are used to search in solution space in the Bunch method [5, 7 & 21]. In the present approach, the answers space size is nn (n represents the number of the artifacts) with the majority of them indicating a similar modularization. Bushehiran and Parsa presented coding of DAGC [15] to resolve the present study, which decreases the state's space to n!. Tajgardan et al. [22] have developed a method based on the EDA ("Estimation of Distribution Algorithms") that does not need the specification of GA parameters. Izadkhah et a1. [16] proposed the E-CDGM technique in which 1st transforms the source code into intermediate code named mCode from CDG ("Call Dependency Graph") and recommends a modularization using fitness function (employing method to method relations, class method, &class-property) and DAGC encoding and self-automata method. The MaABC method, established by Amarjeet et al. [23], is a MOO ("Multi-Objective Optimization") technique on the basis of the bee population approach for software modularization. +ey also offered PSOMC [24],a PSO-based clustering module that divides software systems with optimizing intra-cluster reliance, number of modules per cluster, clusters number, and inter-cluster reliance.

In the background of different programming languages like Pascal,C, and COBOL, the software remodularization research study is comparatively deep-rooted. Though, to cope with the difficulty of current schemes, particularly those established in OOP languages it still needs groundbreaking methods [12]. To enhance the program design, several studies in the outline of object-oriented software were planned for modularizing the classes into packages/modules. The reports[12, 13, 18, 21, 26] communicate that software remodularization is avital and stimulating issue in the arena of software engineering. Several remodularization techniques were suggested by investigators and academics in the domain

of “software engineering” during the last 2eras. When it comes to software remodularization, Wiggerts [37] was the first to create a theoretical framework. They offered a remodularization issue as a software clustering issue that might be resolved via employing clustering methods and cluster assessment standards. They deliberated numerous software entity similarity criteria for clustering assessment and provided an immediate of relevant clustering techniques for clustering methods. Far along various deterministic as well as intelligence-based methods to resolve this issue as a clustering issue was explored in recent years (example [1, 3, 5, 10, 11, 25,27])

Most of the present methods conducted software remodularization from scrape instead of enhancing the current software modular assembly. Few studies have addressed the issue of software remodularization inside the context of current software rottenness [3, 18]. Lately, “Abdeen et al.” [4] suggested a “single-objective software remodularization” method on the basis of the “SA” (Simulated Annealing) method. Their method intended to decrease the “package coupling” and recover the “package” cohesiveness by transferring the classes to current “packages”. Though, SA may enhance some impartial on the additional parameter’s cost as a single remodularization method.

To discourse such boundaries, similar authors [23] presented a MOO for remodularization on the current package group. While the method is challenging and real, they have utilized the partial features of dealings conducive to the linking amid software basics. Stimulated with the software remodularization method in “Abdeen et al.” [13], we suggest “search-based remodularization” to reorganise the packages in their existing state The suggested technique guarantees that the optimization is carried out with as little change to the existing package association as is practically possible. This strategy has the benefit of keeping the remodularization cost low.

Description of the Problem

Upholders entail a broad variety of software fundamental structural evidence for manipulating the diverse quality standards to remodularize the software system. The official data like the association with their asset amongst classes is extensively utilized. Legitimately, the denotation of remodularization issue is to advance the stated modularity eminence standards of a prevailing package association of an object-oriented software structure with respect to the smallest conceivable agitation.

The following assumptions limit package structure, class coupling strength,and class connections in the remodularization issue:

(a) An OO system's package is stated as a module, as well as classes are signified as software objects.
(b) The module is well-known as a coherent collection of classes, implying that every class included inside a package is strongly connected.
(c) In the following modularization resolution, all classes within a software system should be confined to only one package, which is a significant constraint to consider.
(d) The number of packages doesn’t rise or decline during remodularization. The classes throughout the organization may be linked using structural association.
(e) The associations may be unweighted orbiased.

Remodularization Process

The purpose of this research article was to develop and create a "object-oriented package assembly" method with the goal of increasing the quality of the current package assembly of the object-oriented scheme as a consequence. In specific, the chief area of the planned method is to progress modularity excellence, mainly the MQ measure of the current package assembly with negligible probable alteration in the “unique package” group.

To attain the aim, the “software remodularization” issue is framed as a SOO (“Single-Objective Optimization”) and

MOO approachin which the software modularity is enhanced along with the minutest feasible trepidation in the present modular assembly with the assistance of GA. Precisely, the optimization procedure of the GA is measured by joining a reprimanded fitness function, i.e., PMQ. PMQ is used in SOO, while in MOO, the same PMQ is enhanced by secondary quality criteria like these:

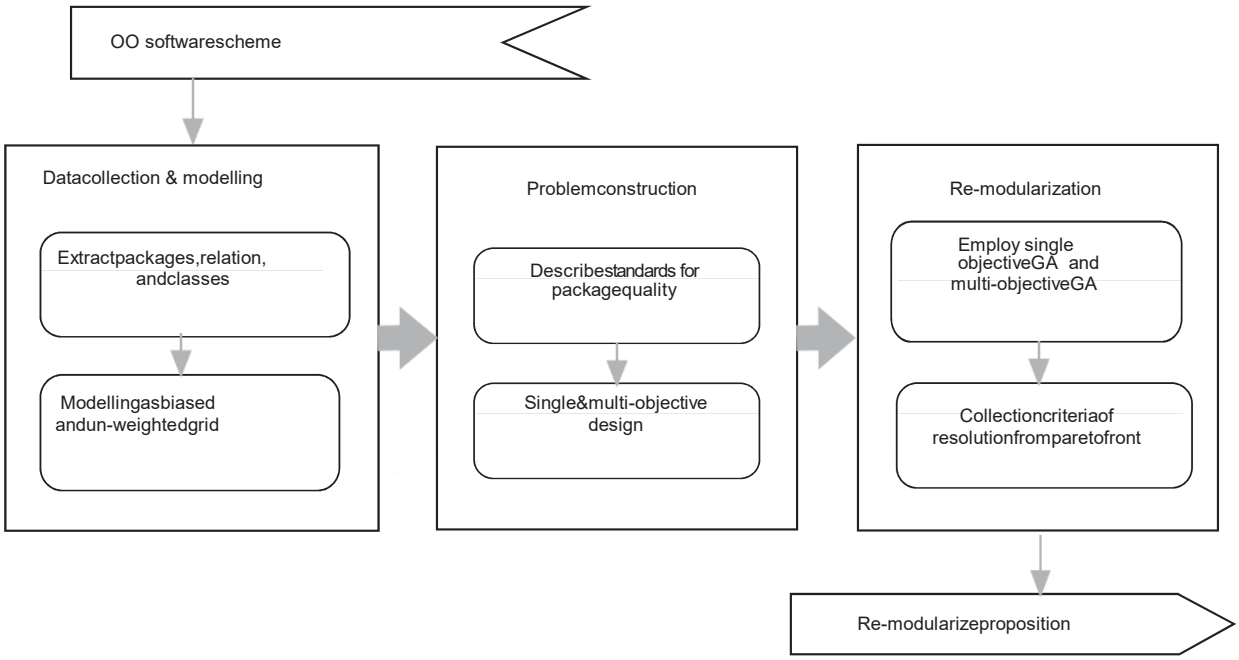

Fig.4: Software Remodularization Method Representation of the Process

(1) maximize package cohesion;

(2) maximize MQ;

(3) minimize package coupling.

The secondary objective meanings are second-hand to simply direct the optimization procedure to a healthier primary objective. The overall assembly of "software remodularization" is exemplified in Fig 4. The International Organization for Standardization has awarded this organisation an I/P for a unique package group of the Objective Optimization software system, as well as the much-debated impartial standards (ISO).

The software strategy must be specified in a manner that allows for the use of a large number of operators in order to frame the remodularization issue as a search-based optimization problem. We present an approach that employs both an unweighted graph (Gu) and a one-sided graph in this study (Gw). "The one-sided graph Gw is well-known as a three-tuple "Gw=(V, E, W)", here V={v0,v1,vn} which is the collection of apexes where every apex signifies a class, "E={e(vi,vj)}⊂V×V" indicates The collection of focused edges, which includes every edge, and W represent the collection of weights for every edge, which may have a real value based on the connections between various classes, describe the construction in two classes. "The Gu (unweighted graph) is also well-known as a two-tuple Gw=(V, E), here V well as E has the similar meaning as in the one-sided graph".

At least one link between the two classes may be shown by the occurrence of an edge.

In an object-oriented software classification, a class may be related to an additional class by zero or more relationships with distinct categories. Connection refers to such a link. The weight of a link is determined by taking into account three factors:

(1) kinds of relations;
(2) weights of each category of relations.
(3) number of examples of relations;

Amarjeet and Chhabra [5, 6, 19] explored the eight well-known connections [such asThrows (TH),Return (RE),Is of Type (IT),Implement (IM),Calls (CA]),Reference (RE),Has Parameter (HP), extends (EX)] in this research article.

Fitness Function

To redirect the optimization procedure of the GA to a better modularization resolution displaying the least difference from the current modular construction, a satisfactory "fitness function" is mandatory.

"In this research article, we describe a different "fitness function", specifically "PMQ", in which the modularity excellence condition", such as the metric of MQ, is reprimanded with the metric of MoJoFM. The PMQ, as well as MQ metrics, are described as follows:

MQ: It is intended to assess the software products modularly. "It is expressed as the total of MFs ("Modularization Factors") and MF is restrained in relation to the inter and intra-package coupling".

MQ = ∑n k=1 MFk where MFk = Σ 0|I/ i+1/2 j if i > 0……(i)

here n signifies the entire number of packages,i &jindicates the intra and inter-package coupling

MQ displays a trade-off between cohesion and coupling. The other metric like elementary MQ [25] may be utilized to assess the modularity. The main difficulty of elementary MQ is that it could not be utilized to count the degree of the quality of modularization resolutiongained from biased edges graphs.

the MQ metric is redefined in PMQ by increasing PD ("Perturbation Degree") as a consequence. The PD is well-known in relation to the movingand joiningof the remodularization process and It's a consequence of the MoJoFM measurement [36].

"PMQ=MQ×PD where PD=1−mno(Mnew, Morg)"/max(mno(∀Mnew, Morg))……..(ii)

"In this case, mno (Mnew,Morg) denotes the smallest number of Join or Move processes required to change the modularization answer M new to the modularization resolution Morg & max (mno(Mnew,Morg)) denotes the greatest probable distance between any novel modularization key Mnew and the unique modularization result Morg. The MoJoFM measure was used to calculate the relationship in two modularization outcomes".

Other resemblance metrics used in the study include a2a ("Architecture-to-Architecture") [24] and cluster-to-cluster coverage [20]. Though, in this case, the MoJoFM measure is suitable than a2a and c2ccvg. When the software remodularization methods being evaluated have the same classes (as in the example), a2a & c2ccvg would provide consequences with a narrow range of variance, making it impossible to distinguish between the remodularization methods. As a result, the MoJoFM measure is preferable to a2a and c2ccvg in our scenario.

Multi-Objective Remodularization Process

This technique optimizes more than one impartial. It regulates a customary of modularizations

"M*for which F(M*)=min/max(F1(M), F2(M), . . . , Fm(M))|M∈ψ" ……(iii)

here m represents the sum of "objective functions",ψ indicates the set of all possible modularizations and Fi denotes the ith objective function.

There is usually no one best resolution in "multi-objective software optimization", although there may be many non-dominated modularization keys.

"if and only if2 modularization explanations "M1,M2∈ψ", "M1-solution" is shown to "M2-dominate solution" (signified as "M1≤M2)

∀i∈(1, . . . , m) Fi (M1)≤ Fi (M2)∧∃∈(1, . . . , m)Fi(M1)<Fi (M2)……(iv)"

Then, "M1, as well as M2, are considered to be non-dominated answers". The front of the Pareto is a collection of all non-dominant explanations in objective space. Multi-objective modularization approaches offer flexible modularization results in which the programmer has additional choices for selecting the optimal solution depending on their needs. A MOO strategy is used in order to recover the PMQ function of single-objective with the aid of additional objective functions. The software's MQ value enhances more as MOO with the supply of additional conflicting impartial functions like cohesiveness and coupling, as likened to development via SOO, as shown by Pradiwong et al. [11].

Software Process and Algorithmic Limits

The test examines the proposed method's application to 5 completely open Objective Optimization software packages on the basis of Java programming, as well as two haphazard systems. JavaCC, XML API DOM, Java Servlet API, and JUnit, are examples of real-world systems, whereas Test50, Test100, and Test100 are examples of arbitrary systems. The software systems are modelled into 2 categories. The 1st category is a biased paradigm in which the edge mass is allocated rendering to the technique debated in the aforementioned part.

The 2nd category is an unweighted prototypical which assigns a binary number to edge weight. Table 1 provides further information on the examples of designated issues. We chose this OO software for our evaluation because they feature a wide variety of classes and packages and varying levels of complexity. A software system's various sizes and problems might give a clear vision into modularization strategies. It aids in the reduction of bias in the outcomes. Other earlier researchers have utilized these systems to assess their solutions for the remodularization process incomparable issues. Algorithmic Parameters 5.2 in this article, the SGA is cast-off for SOOas well as the NSGA-II for MOO. The NSGA-II is a GA on the basis of non-domination categorization notions of the MOO approach. A collection of non-dominant solutions, called the Pareto set, is generated by the algorithm.

For these GAs, the same limit configuration is employed in this research study as it is in the literature [8, 17, 21]. As following is the restriction values:

(1) the highest generations set has 200times the number of classes (N),

(2) The chance of crossing is established at 0.8, whereas the chance of mutation is set at 0.004log2(N),

(3) unvarying mutation operator and single-point crossover operative,

(4) The population scope is ten times the sum of classes (N).

Valuation Criteria for the Result

We utilize the MQ focuses in evaluating the superiority of modularization and the degree per refactoring of reached enhancement to evaluate the agitation acquired by the planned method to measure the answers gained by the planned strategy. The following is the definition of the RRAI in terms of the MQ dimension:

"RRAI(MQ)=RPMC(MQ)"/RPC(MQ)……
……(v)

The "rate/class" of the MQ dimension is denoted by RPC(MQ), which is determined as follows:

"RPC(MQ)=MQ"or/|C|……
…….(vi)

here MQor indicates the MQ cost of the software system's unique structure

C denotes the established structure of each class.

The rate/enthused class of the MQ dimension is represented by the RPMC(MQ), which is calculated as follows:

"RPMC(MQ)=ΔMQ
NC……..(vi i)"

In new modularization, ΔMQ denotes the augmented MQ value in modularization, while NC denotes the classes number that alters packages. The greater the "RRAI(MQ)"value, the lesser the alteration with admiration to the MQ dimension.

Analysis and Results

This segment grants the analysis and findings of the research that liken the values of MQ derived from penalized as well as global optimization in every situation. Table 1 depicts the outcomes of the median, mean & standard eccentricity of MQ sets formed using MGA and SGA in comprehensive optimization across unweighted and biased software processes.

Problems	MGA			SGA			MGA vs. SGA		
	Mean	Median	Standard deviation	Mean	Median	Standard deviation	Δ	*p*-Value	Winner
Unweighted									
JavaCC-1.5	4.007	4.132	0.015	3.854	4.051	0.062	+0.081	0.862	≈
Junit-3.8.1	4.298	4.325	0.013	4.224	3.621	0.027	+0.704	0.012	⟵
Servlet API-2.3	3.547	3.621	0.023	3.407	3.687	0.046	−0.066	0.394	≈
XML API-1.0.b2	8.249	8.327	0.063	8.160	7.214	0.055	+1.113	0.001	⟵
DOM 4J-1.5.2	6.557	6.471	0.049	6.752	6.453	0.042	+0.018	0.256	≈
Random 50	4.077	5.018	0.075	4.019	4.034	0.089	+0.984	0.112	≈
Random 100	6.230	6.501	0.053	5.997	5.674	0.081	+0.927	0.135	≈
Random 150	7.358	7.234	0.081	7.138	8.443	0.039	−1.209	0.002	⟶
Weighted									
JavaCC-1.5	6.607	6.213	0.045	6.096	6.151	0.086	+0.062	0.024	≈
Junit-3.8.1	6.455	6.316	0.072	6.061	5.281	0.071	+1.035	0.006	⟵
Servlet API-2.3	6.574	6.415	0.047	5.113	5.211	0.054	+1.204	0.015	⟵
XML API-1.0.b2	8.721	8.101	0.037	7.123	8.015	0.046	+0.086	0.637	≈
DOM 4J-1.5.2	9.754	10.031	0.079	8.170	8.042	0.081	+1.989	0.001	⟵
Random 50	6.356	6.581	0.098	5.901	5.312	0.069	+1.269	0.006	⟵
Random 100	8.316	8.531	0.065	7.895	7.154	0.097	+1.377	0.008	⟵
Random 150	9.244	9.643	0.104	9.020	8.179	0.071	+1.464	0.017	⟵

Table1:Global Optimization Attained Median, Mean, &Standard Deviation of MQ Measure

In consideration of Single-Objective GA versus Multi-Objective GA; a few of the analyses have been shared in the below section.

In this segment, we examine the MQ values shaped with MGA and SGA across unweighted as well as biased software systems. The following is a summary of the comprehensive study:

(1) unweighted and global optimization process: the outcomes exhibited in Table 1 display that "the MGA achieves improved than the SGA in 6 out of 8 circumstances with 2 cases are knowingly restored";

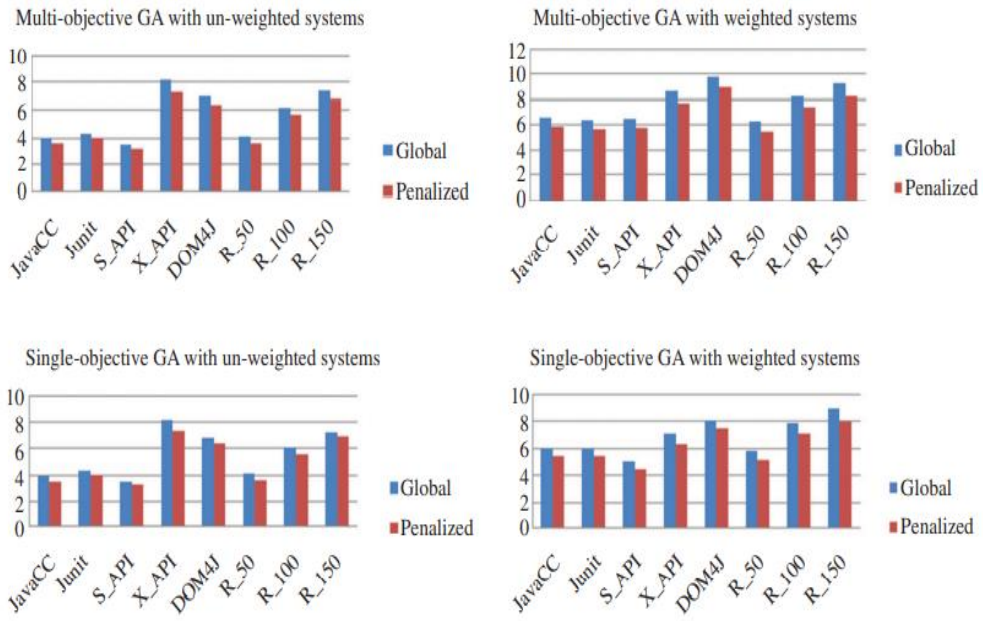

Fig:5:MQ Values study in Global Optimization through Penalized Optimization

(2) weighted and global optimization process: Table 1 depicts the MGA outperforms the SGA throughout all issue scenarios, with 6 cases being considerably superior.

(3) unweighted and penalized optimization process: Table 1 indicates that the MGA outperforms the SGA in 6 of the 8 circumstances when one case is suggested to be restored; The symbol ‘’signifies the instances in which the MGA showed the greater performance inside the pairwise “Wilcoxon” trail at 95 percent implication level(α is equal to 0.05); “the symbol” ‘⟵’ denotes the instances in which the “SGA displayed the larger performance”, as well as the symbol ‘≈’,signifies instances in where there is no significant variance amongst the SGA and MGA. Values of delta Δ signify the change amongst the median sets of SGA and MGA.

Conclusion & Future Works

The purpose of this study effort was to provide a novel technique for object-oriented software remodularization, with the goal of increasing the software quality of the current package association while minimising any possible hazards. This software rearrangement displaying smaller agitation is extremely valuable for upholders to find an important development in modularization superiority of the software without choosing for the whole remodularization which could be very expensive, time-consuming, and hard to understand.

A PMQ measure was created as a fitness function to achieve the aim, similar to the original MoJoFM and MQ metrics. The method was assessed on 8 real and arbitrary weighted as well as unweighted software systems. The found findings offered adequate empirical data that the future method is capable of recovering the current package group quality by adapting the original package group as few as feasible. The implications of the findings are that we may achieve the same degree of eminence with much smaller agitations as we can with the overall remodularization. Henceforward it could be decided that the method projected in this research article is very helpful for the engineering community to advance software structural quality with smaller time and cost. The major curb of the study is the examination of the GA reduces if the number of objectives upsurges by more than three. Multi-objective GAs may be used to overcome this constraint. Future work in this course might incorporate other goals and limitations, such as improving

package structures with less agitation so that this movement could be utilized more often during software maintenance.

References

[1] A. Adolfsson, M. Ackerman, and N. C. Brownstein, “To cluster, or not to cluster: An analysis of clusterability methods,” Pattern Recognition, vol. 88, pp. 13–26, 2019.

[2] P. Antonellis, D. Antoniou, Y. Kanellopoulos, C. Makris,E. Theodoridis, C. Tjortjis, and N. Tsirakis, “Clustering for Monitoring Software Systems Maintainability Evolution,” Electronic Notes in Theoretical Computer Science, vol. 233, pp. 43–57, 2009.

[3] G. ¸Serban and I.-G. Czibula, “Object-Oriented Software Systems Restructuring through Clustering,” in International Conference on Artificial Intelligence and Soft Computing (ICAISC), 2008, pp. 693–704.

[4] J. Feng and H. Seok, “Applying agglomerative hierarchical clustering algorithms to component identification for legacy systems,” Information and Software Technology, vol. 53, no. 6, pp. 601–614, 2011.

[5] R. A. Bittencourt and D. D. S. Guerrero, “Comparison of Graph Clustering Algorithms for Recovering Software Architecture Module Views,” in the 13th European Conference on Software Maintenance and Reengineering. IEEE,2009, pp. 251–254.

[6] B. Joshi, P. Budhathoki, W. L. Woon, and D. Svetinovic, “Software Clone Detection Using Clustering Approach,”in International Conference on Neural Information Processing, 2015, pp. 520–527.

[7] R. Naseem, A. Ahmed, S. U. Khan, M. Saqib, and M. Habib, “Program restructuring using agglomerative clustering technique based on binary features,” in 2012 International Conference on Emerging Technologies. IEEE,2012, pp. 1–6.

[8] M. Kargar, A. Isazadeh, and H. Izadkhah, “Multiprogramming language software systems modularization,” Computers and Electrical Engineering, vol. 80, pp. 1–22, 2019.

[9] Y. Yu, J. Lu, J. Fernandez-Ramil, and P. Yuan, “Comparing web services with other software components,” in IEEE International Conference on Web Services (ICWS), 2007, pp. 388–397.

[10] N. Arunachalam, A. Amuthan, C. Kavya, M. Sharmilla, K. Ushanandhini, and M. Shanmughapriya, “A survey on web service clustering,” in 2017 International Conference on Computation of Power, Energy Information and Communication (ICCPEIC), 2017, pp. 247–252

[11] P.AmarjeetandJ.K.Chhabra,FP-ABC:Fuzzy-Paretodominancedrivenartificialbeecolonyalgorithmformany-objectivesoftwaremoduleclustering,Comput.Lang.Syst.Struct.**51**(2018),1–21.

[12] .Anquetil,S.Denier,S.Ducasse,J.Laval,andD.Pollet,Software(re)modularization:fightagainstthestructureerosionandmigrationpreparation,2010.

[13] .AnquetilandT.C.Lethbridge,Experimentswithclusteringasasoftwaremodularizationmethod,in:WorkingConferenceonReverseEngineering,pp.235–255,IEEECSPress,Washington,DC,USA,1999.

[14] .AnquetilandT.C.Lethbridge,Comparativestudyofclusteringalgorithmsandabstractrepresentationsforsoftwarere-modularization,IEEProc.Softw.**150**(2003),185–201.

[15] .F.Ardabili,Computationalintelligenceapproachformodelinghydrogenproduction:areview,Eng.Appl.Comput.FluidMech.**12**(2018),438–458.

[16] M. Barros, An analysis of the effects of composite objectives in multiobjective software module clustering, in: Proceedingsofthe14thInternationalConferenceonGeneticandEvolutionaryGECCC-12,TerenceSoule(Ed.).pp.1205–212,ACM,NewYork,NY,USA,2012.

[17] G.Bavota,A.D.Lucia,A.Marcus,andR.Oliveto,Softwarere-modularizationbasedonstructuralandsemanticmetrics,in:ProceedingsofWCRE’2010,pp.195–204,IEEE,Beverly,Massachusetts,USA,2010.

[18] .Corwin,D.F.Bacon,D.Grove,andC.Murthy,MJ:Arationalmodulesystemforjavaanditsapplications,in:Proceedings of the 18th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications,pp.241–254,NewYork,NY,USA,2003.

[19] .Deb,A.Pratap,S.Agarwal,andT.Meyarivan,Afastandelitistmultiobjectivegeneticalgorithm:NSGA-II.IEEETrans.Evolut.Comput.**6**(2002),182–197.

[20] J.Garcia,D.Le,D.Link,A.S.PooyanBehnamghader,E.F.OrtizandN.Medvidovic,Anempiricalstudyofarchitecturalchangeanddecayinopen-sourcesoftwaresystems,Tech.Rep.USC-CSSE.2014.

[21] V.GuptaandJ.K.Chhabra,PackageCouplingmeasurementinobject-orientedsoftware.J.Comput.Sci.Technol.**24**(2009),273–283.

[22] Y.IchisugiandA.Tanaka,Difference-basedmodules:aclassindependentmodulemechanism,in:ProceedingsECOOP2002,2374ofLNCS,SpringerVerlag,Malaga,Spain,2002.

[23] S.M.R.Kazemi,Novelgenetic-basednegativecorrelationlearningforestimatingsoiltemperature,Eng.Appl.Comput.FluidMech.**12**(2018),506–516.

[24] .Le,P.Behnamghader,J.Garcia,D.Link,A.ShahbazianandN.Medvidovic,Anempiricalstudyofarchitecturalchangeinopensourcesoftwaresystems,TechnicalReportUSC-CSSE-2014-509,CenterforSystemsandSoftwareEngineering,UniversityofSouthernCalifornia,2014.

[25] S. Mancoridis, B. S. Mitchell, C. Rorres, Y. F. Chen and E. R. Gansner, Using automatic clustering to produce high-levelsystemorganizationsofsourcecode,in:Proceedings.6thInternational WorkshoponProgramComprehension.IWPC’98,pp.45–53,IEEE,Ischia,Italy,1998.

[26] .Mancoridis,B.S.Mitchell,C.Rorres,Y.F.ChenandE.R.Gansner,Bunch:recoveryandmaintenanceofsoftwaresystemstructures,in:ProceedingsoftheIEEEInternationalConferenceonSoftwareMaintenance,pp.50–59,IEEE,Oxford,UK,1999.

[27] S. McDirmid, M. Flatt and W. Hsieh, Jiazzi: new age components for old fashioned Java, in: Proceedings of the 16th ACMSIGPLANConferenceonObject-OrientedProgramming,Systems,Languages,andApplications(OOPSLA’01),pp.211–222,ACM,NewYork,NY,USA,2001.

[28] .MeltonandE.Tempero,TheCRSSmetricforpackagedesignquality,in:ProceedingsofACSC’2007,pp.201–210,AustralianComputerSocietyInc.,Darlinghurst,Australia,2007.

[29] .S.MitchellandS.Mancoridis,Ontheautomaticmodularizationofsoftwaresystemsusingthebunchtool,IEEETrans.Softw.Eng.**32**(2006),193–208.

[30] .Moazenzadeh,Couplingafireflyalgorithmwithsupportvectorregressiontopredictevaporationinnorthern Iran,Eng.Appl.Comput.FluidMech.**12**(2018),584–597.

[31] .Praditwong,M.HarmanandX.Yao,Softwaremoduleclusteringasamultiobjectivesearchproblem,IEEETrans.Softw.Eng.**37**(2011),264–282.

[32] https://github.com/SoftwareMaintenanceLab